

Design of Mahimahi-Inspired Congestion-Control Environments

1 Overview

The repository defines four scenario-shaped Gymnasium environments for congestion-control experiments:

- (1) a stable wired bottleneck,
- (2) a cellular trace with bursty capacity variation,
- (3) a LEO-inspired trace with abrupt delay and bandwidth changes, and
- (4) a multi-flow fairness stress test.

Each scenario defaults to a fast in-process simulator, but can also be run through the real `mm-link` subprocess adapter by passing `use_mahimahi=True`. In simulator mode, the environments model the same core signals that Mahimahi-style experiments expose: bandwidth, propagation delay, queueing delay, packet loss, throughput, and multi-flow sharing. In real Mahimahi mode, the scenario classes delegate to scenario-shaped wrappers around `MahimahiSubprocessEnv`, preserving the same action and observation interfaces while launching `mm-link` with scenario-specific trace files.

2 Shared Single-Flow Design

The wired, cellular, and LEO environments share a single-flow bottleneck model in simulator mode. At each step, the agent chooses a target sending rate:

$$a_t = r_t \quad \text{in Mbps.}$$

The environment replays a link trace with a capacity C_t , propagation delay D_t , and fixed step duration. If the selected sending rate exceeds the available service rate, data accumulates in a finite bottleneck queue. Once the queue is full, excess traffic is dropped.

The single-flow observation is seven-dimensional:

$$o_t = \left[\frac{r_t}{r_{\max}}, \frac{x_t}{r_{\max}}, \frac{C_t}{r_{\max}}, \frac{D_t}{D_{\max}}, \frac{Q_t}{Q_{\max}}, L_t, B_t \right],$$

where r_t is the chosen send rate, x_t is delivered throughput, C_t is link capacity, D_t is propagation delay, Q_t is queueing delay, L_t is loss fraction, and B_t is normalized queue occupancy.

The reward is:

$$R_t = x_t - \lambda_D(D_t + Q_t) - \lambda_L L_t.$$

This reward encourages high delivered throughput while penalizing round-trip latency and loss. The exact delay and loss penalties can vary by scenario. The default single-flow loss penalty is $\lambda_L = 10.0$. The default delay penalty is $\lambda_D = 0.015$, except for the LEO-inspired scenario, which lowers it to $\lambda_D = 0.012$.

2.1 Optional MPCC-Style Reward

The scenarios also support an optional MPCC-style reward by setting `reward_mode=mpcc`. The default reward above remains available as `reward_mode=default`; MPCC mode changes only the scalar reward and diagnostic fields in `info`, not the action or observation spaces.

The MPCC reward uses smoothed measurements $\hat{T}$ and $\hat{R}$ for throughput and RTT. The simulator updates these smoothed values with first-order filters using configurable time constants τ_T and τ_R . In single-flow simulator mode, R_0 is the current propagation delay. In the multi-flow simulator, which has no explicit propagation-delay trace, R_0 defaults to 20 ms and can be overridden with `mpcc_base_rtt_ms`.

The reference curve in throughput–RTT space is:

$$\Gamma(\theta) = (C\theta, R_0 + \alpha\theta^2), \quad \theta = \text{clip}\left(\frac{\hat{T}}{C}, 0, 1\right).$$

The unit tangent is:

$$\mathbf{t}(\theta) = \frac{(C, 2\alpha\theta)}{\sqrt{C^2 + 4\alpha^2\theta^2}} = (t_T, t_R).$$

Given $\Gamma(\theta) = (\Gamma_T, \Gamma_R)$, the contouring and lag errors are:

$$e_c = t_R(\hat{T} - \Gamma_T) - t_T(\hat{R} - \Gamma_R),$$

$$e_\ell = t_T(\hat{T} - \Gamma_T) + t_R(\hat{R} - \Gamma_R).$$

The MPCC stage cost implemented by the environments is:

$$\ell_{\text{mpcc}} = w_c e_c^2 + w_\ell e_\ell^2 + w_d [\hat{R} - R^*]_+^2 + w_u \frac{\sum_i (s_{i,t} - s_{i,t-1})^2}{s_{\max}^2} + w_{\text{loss}} L_t^2 - w_p \frac{\hat{T}/C}{\hat{R}/R_0}.$$

Gymnasium maximizes reward, so the environment returns:

$$R_t^{\text{mpcc}} = -\ell_{\text{mpcc}}.$$

The default MPCC weights are intentionally modest because the geometric errors are measured in physical units:

$$w_c = 10^{-3}, \quad w_\ell = 10^{-3}, \quad w_d = 10^{-4}, \quad w_p = 1.0, \quad w_u = 0.1, \quad w_{\text{loss}} = 0.$$

If α is not provided, it defaults to half the queue-delay scale. If R^* is not provided, it defaults to R_0 plus one quarter of the queue-delay scale.

The `info` dictionary exposes MPCC terms for debugging and analysis:

- `mpcc_stage_cost`, `mpcc_contour_error`, and `mpcc_lag_error`;
- `mpcc_power`, `mpcc_smoothing_penalty`, and `mpcc_theta`;
- `mpcc_smoothed_throughput_mbps` and `mpcc_smoothed_rtt_ms`.

2.2 Real Mahimahi Single-Flow Mode

When `use_mahimahi=True`, the single-flow scenarios instantiate `SingleFlowMahimahiScenarioEnv`. This wrapper delegates stepping to `MahimahiSubprocessEnv` with one action dimension, then maps the adapter’s measurement dictionary back into the seven-dimensional scenario observation. Since a real controller may report queueing delay without explicit queue occupancy, the final queue-related observation slot is represented by normalized queueing delay in this mode.

The default controller command is:

```
python examples/mm_link_json_controller.py
```

The wrapper forwards `controller_cmd`, `mahimahi_cmd`, `timeout_s`, `restart_on_reset`, and `render_mode` to the subprocess adapter. In `reward_mode=mpcc`, the wrapper replaces the adapter’s default reward with the same MPCC reward calculation using the measurements reported by the controller.

3 Stable Wired Bottleneck

Environment ID: `MahiWiredBottleneck-v0`

Class: `WiredBottleneckEnv`

3.1 Design Goal

This environment is a stationary sanity-check task. The optimal behavior should be easy to understand: send near the fixed bottleneck rate, avoid persistent oversending, and keep delay/loss low.

3.2 Configuration

- Capacity: 10 Mbps
- Propagation delay: 20 ms
- Step duration: 100 ms
- Queue size: 150 ms
- Maximum send rate: 20 Mbps
- Real Mahimahi traces: `mahimahi/12mbps.trace` for both uplink and downlink

3.3 Rationale

A fixed 10 Mbps link creates a clean, stationary bottleneck. The low 20 ms propagation delay resembles a wired path rather than a cellular or satellite path. The queue is intentionally modest: large enough to permit transient overshoot, but small enough that persistent oversending produces visible queueing delay and loss. The 20 Mbps maximum action lets the agent overshoot by a factor of two, so it can learn the cost of aggression.

3.4 Expected Policy

A good policy should converge to sending close to 10 Mbps, with low loss and low queueing delay.

4 Cellular Bursty Capacity

Environment ID: MahiCellularBursty-v0

Class: CellularBurstyEnv

4.1 Design Goal

This environment represents a cellular link where radio conditions cause sharp capacity variation. The agent must adapt to bursts and collapses without filling the queue.

4.2 Capacity Trace

[8.0, 7.5, 3.0, 1.2, 2.0, 12.0, 15.0, 6.0, 4.0, 2.2, 1.0, 9.0, 14.0, 18.0, 5.0, 2.5, 1.5, 7.0, 11.0, 4.5]

4.3 Delay Trace

[55, 58, 70, 95, 85, 60, 50, 65, 78, 90, 110, 62, 54, 48, 72, 88, 105, 68, 58, 76]

4.4 Rationale

Capacity ranges from roughly 1 Mbps to 18 Mbps. Low-capacity periods punish aggressive sending, while high-capacity bursts reward rapid utilization. The delay trace rises during weaker-capacity periods to mimic cellular instability and buffer pressure. The queue size is 300 ms, larger than the wired queue, because cellular paths often exhibit deeper and more variable buffering.

In real Mahimahi mode, this scenario uses `mahimahi/traces/TMobile-LTE-driving.up` as the default uplink trace and `mahimahi/traces/TMobile-LTE-driving.down` as the default downlink trace. The default maximum send rate is 30 Mbps.

4.5 Expected Policy

A good policy should increase its sending rate during high-capacity bursts and back off during collapses, while avoiding persistent cellular queue buildup.

5 LEO-Inspired Satellite Trace

Environment ID: MahiLeoSatellite-v0

Class: LeoSatelliteEnv

5.1 Design Goal

This environment models abrupt bandwidth and propagation-delay shifts inspired by low-Earth-orbit satellite networking. It is not a detailed orbital model; the goal is to stress robustness to sudden regime changes.

5.2 Capacity Trace

[35, 35, 32, 8, 6, 45, 48, 12, 10, 9, 50, 52, 18, 15, 4, 4, 30, 42, 46, 7]

5.3 Delay Trace

[35, 36, 38, 80, 95, 42, 36, 75, 88, 92, 40, 34, 65, 70, 120, 125, 55, 44, 38, 100]

5.4 Rationale

The capacity trace includes high-throughput windows near 50 Mbps and sudden collapses to 4–8 Mbps. The delay trace includes abrupt jumps from roughly 35 ms to more than 120 ms. These shifts are meant to mimic the kind of behavior that can arise from satellite handoff, path geometry changes, obstruction, or beam transitions.

The delay penalty is slightly lower than in some other scenarios:

$$\lambda_D = 0.012.$$

This prevents the policy from becoming overly conservative in a setting where propagation delay variation is inherent.

In real Mahimahi mode, this scenario uses `mahimahi/traces/Verizon-LTE-short.up` as the default uplink trace and `mahimahi/traces/Verizon-LTE-short.down` as the default downlink trace. The default maximum send rate is 60 Mbps.

5.5 Expected Policy

A good policy should exploit high-capacity windows while remaining robust to sudden bandwidth drops and delay jumps.

6 Multi-Flow Fairness Stress Test

Environment ID: `MahMultiFlowFairness-v0`

Class: `MultiFlowFairnessEnv`

6.1 Design Goal

This environment tests whether a controller can balance total throughput and fairness when multiple flows share a single bottleneck.

6.2 Default Configuration

- Number of flows: 3
- Shared bottleneck capacity: 12 Mbps
- Maximum send rate per flow: 15 Mbps
- Queue size: 200 ms
- Real Mahimahi traces: `mahimahi/12mbps.trace` for both uplink and downlink

6.3 Action

The action is a vector of per-flow sending rates:

$$a_t = [r_{1,t}, r_{2,t}, r_{3,t}].$$

6.4 Observation

For N flows, the observation is:

$$o_t = \left[\frac{r_{1,t}}{r_{\max}}, \dots, \frac{r_{N,t}}{r_{\max}}, \frac{x_{1,t}}{r_{\max}}, \dots, \frac{x_{N,t}}{r_{\max}}, \frac{C}{r_{\max}}, B_t, L_t, J_t \right],$$

where J_t is Jain's fairness index.

In simulator mode, B_t is normalized queue occupancy. In real Mahimahi mode, the corresponding slot is normalized queueing delay, because the subprocess adapter may not receive an explicit queue occupancy measurement from the controller.

6.5 Reward

The reward combines total throughput, fairness, delay, and loss:

$$R_t = \sum_{i=1}^N x_{i,t} + \lambda_F J_t - \lambda_D Q_t - \lambda_L L_t.$$

The default fairness weight is:

$$\lambda_F = 8.0.$$

The default delay and loss penalties are $\lambda_D = 0.015$ and $\lambda_L = 4.0$.

Jain's fairness index is:

$$J_t = \frac{(\sum_i x_{i,t})^2}{N \sum_i x_{i,t}^2}.$$

In `reward_mode=mpcc`, the multi-flow environment applies the same throughput-RTT MPCC objective to aggregate throughput and shared queueing delay. It additionally adds a fair-share penalty:

$$\ell_{\text{fair}} = w_f \sum_{i=1}^N \left(s_{i,t} - \frac{C}{N} \right)^2.$$

This term biases each sender toward its nominal share of the bottleneck capacity. The default MPCC fair-share weight is:

$$w_f = 10^{-2}.$$

The total MPCC reward for the multi-flow scenario is:

$$R_t^{\text{mpcc,multi}} = -(\ell_{\text{mpcc}} + \ell_{\text{fair}}),$$

where ℓ_{mpcc} includes the contouring, lag, delay, power, smoothing, and optional loss terms defined in the shared MPCC reward section. The `info` dictionary includes `mpcc_fairness_penalty` in this mode.

6.6 Rationale

If the reward only used total throughput, an agent could learn unfair allocations where one flow dominates. Jain’s fairness term rewards equal sharing. Since each flow can request up to 15 Mbps but the shared bottleneck is only 12 Mbps, the combined action can dramatically oversubscribe the link. This forces the agent to learn both rate control and fair allocation.

6.7 Expected Policy

A good policy should keep aggregate sending near the shared capacity, distribute throughput roughly equally across flows, maintain high Jain fairness, and avoid queue buildup and loss.

6.8 Real Mahimahi Multi-Flow Mode

When `use_mahimahi=True`, `MultiFlowFairnessEnv` delegates to `MultiFlowMahimahiScenarioEnv`. The wrapper configures `MahimahiSubprocessEnv` with `action_dim = N`, then reconstructs per-flow throughput observations from the controller’s `throughputs_mbps` field when available. If only aggregate throughput is reported, the wrapper distributes it across flows in proportion to the chosen send rates before computing Jain’s fairness index. In `reward_mode=mpcc`, the wrapper computes the MPCC reward from the reported aggregate or reconstructed per-flow throughput, queueing delay, RTT, and capacity measurements.

7 Comparison

Environment	Main Challenge	Action Space
<code>MahiWiredBottleneck-v0</code>	Stable bottleneck convergence	One send rate
<code>MahiCellularBursty-v0</code>	Bursty capacity adaptation	One send rate
<code>MahiLeoSatellite-v0</code>	Abrupt bandwidth/delay shifts	One send rate
<code>MahiMultiFlowFairness-v0</code>	Shared bottleneck fairness	Per-flow send rates

Table 1: Summary of the four scenario environments.

8 Real Mahimahi Adapter

The scenario environments use `MahimahiSubprocessEnv` internally when `use_mahimahi=True`. The adapter connects to Mahimahi by launching:

```
mm-link uplink.trace downlink.trace -- <controller command>
```

The Gym environment communicates with the controller process over JSON lines. At each step, the environment writes an action to the controller’s standard input:

```
{"type": "action", "step": 0, "send_rates_mbps": [4.0]}
```

The controller must print one measurement object to standard output:

```
{"throughput_mbps": 3.8, "rtt_ms": 45.0, "queue_delay_ms": 5.0,  
"loss_fraction": 0.0, "capacity_mbps": 10.0}
```

This adapter is intentionally generic. The controller command can be a Python shim, a compiled congestion-control binary, or a script that manages a real sender and receiver inside the Mahimahi shell. It remains an internal building block for the four scenario environments rather than a separately registered Gymnasium environment.

The built-in scenario wrappers use the following default trace files for real Mahimahi runs:

Environment	Uplink trace	Downlink trace
MahiWiredBottleneck-v0	mahimahi/12mbps.trace	mahimahi/12mbps.trace
MahiCellularBursty-v0	mahimahi/traces/ TMobile-LTE-driving.up	mahimahi/traces/ TMobile-LTE-driving.down
MahiLeoSatellite-v0	mahimahi/traces/ Verizon-LTE-short.up	mahimahi/traces/ Verizon-LTE-short.down
MahiMultiFlowFairness-v0	mahimahi/12mbps.trace	mahimahi/12mbps.trace

Table 2: Default real Mahimahi trace files used by the scenario wrappers.

Mahimahi trace reminder: `mm-link` traces are packet-delivery schedules, where each line is a delivery opportunity timestamp for an MTU-sized packet. They are not simple `duration_ms capacity_mbps` rows. The in-process simulator traces use Python arrays or simulator-specific trace formats, while real Mahimahi runs should use Mahimahi-compatible uplink and downlink trace files.

9 Scenario Configuration Direction

A natural next step is to move the remaining hard-coded scenario parameters into configuration files that can be consumed by both the simulator and the real Mahimahi subprocess wrapper. For example:

```
name: cellular_bursty
capacity_trace: ...
delay_trace: ...
queue_size_ms: 300
flows: 1
reward:
  mode: mpcc
  contour_weight: 0.001
  lag_weight: 0.001
  delay_weight: 0.0001
  power_weight: 1.0
  smoothing_weight: 0.1
  fairness_weight: 0.01
```

This would allow policies to be trained quickly in simulation and then evaluated against real `mm-link` experiments with the same scenario definitions.